Solution Architecture

Date	06 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Solution Architecture Diagram:

Presentation / Client Layer

Web App – Loan Application Form (index.html) | Web App – Prediction Result Page (result.html)
Styled with CSS3 (style.css) + Google Fonts (Inter), rendered via Jinja2 templates

↓↑ HTTP Request / Response

API Gateway / Request Handler

Flask Built-in Development Server (localhost:5000)
GET / → renders loan application form
POST /predict → accepts form data, returns prediction
Production deployment: Gunicorn WSGI Server

↓↑ Internal Function Calls

Auth Service

No authentication in the current version.

Future: Session-based login for bank officers.

Core Logic Service

Input validation & form data extraction
LabelEncoder encodes categorical fields (Gender, Education, etc.)
NumPy builds 11-feature array
XGBoost model.pkl runs inference
Returns Approved / Rejected + summary
try/except error handling

External API Integrations

Google Fonts CDN (Inter typeface for UI styling)

No payment or banking API integrated in current version

↓↑ File Read / Model Load

Data / Storage Layer

model.pkl – pre-trained XGBoost model (loaded at server startup via Pickle)
loan.csv – Kaggle dataset (614 records, 13 columns) used for training in train_model.py
No live database – the system is fully stateless

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the loan application form where users input 11 applicant details (Gender, Income, Credit History, etc.). Displays the prediction result page with Approved/Rejected status, applicant summary table, and key decision factors. Handles “Apply Again” and “Review & Resubmit” navigation.	HTML5, CSS3, Jinja2 Template Engine, Google Fonts CDN (Inter)
API Gateway / Request Handler	Acts as the entry point for all HTTP traffic. Handles two routes — GET / renders the input form and POST /predict receives form data, triggers ML inference, and returns the result page. In production, Gunicorn serves as the WSGI layer in front of Flask.	Flask 3.0.3, Python 3.x, Gunicorn (production)
Core Logic Service (ML Engine)	The heart of Smart Lender. Extracts and validates 11 form fields, encodes categorical variables using LabelEncoder, constructs a NumPy feature array, loads the pre-trained XGBoost model from model.pkl, runs prediction inference, and returns the result (1 = Approved, 0 = Rejected) with confidence-based decision factors.	Python 3.x, XGBoost 2.1.0, Scikit-learn 1.5.1 (LabelEncoder), NumPy 1.26.4, Pickle
External API Integrations	Google Fonts API is the only external service used — it loads the Inter typeface via CDN for consistent UI typography. No payment gateway, banking API, or email service is integrated in the current version.	Google Fonts API (CDN)
Data / Storage Layer	Stores the training dataset (loan.csv) used by train_model.py to train and evaluate 4 ML models (Decision Tree, Random Forest, KNN, XGBoost). The best model is serialized as model.pkl and loaded at Flask server startup. No user data is persisted — the system is fully stateless.	CSV File (dataset/loan.csv – 614 rows × 13 columns), Pickle File (model.pkl), Pandas 2.2.2 (for training data processing)